

Hacking Week Report

Maanit Pinaki Trivedi

Fontys University of Applied Sciences

Advanced Cyber Security

Schellekens,Casper

Table of Contents

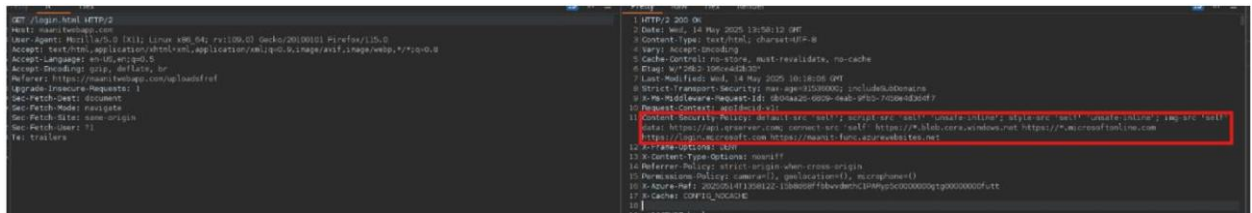
Abstract.....	3
Findings	4
CSP	4
File Upload Vulnerability.....	10
Rate Limiting Bypass	12
Poorly Configured WAF.....	14
Brute Force Bypass.....	16
Bypassing File Upload Rules.....	18

Abstract

- Full stack Node.js app deployed with Azure Web App PaaS
- Main Security: Azure Firewall
- API endpoints for authentication, user management and file upload
- Multi-factor authentication (2FA) using one-time passwords
- JWT-based session management
- Recovery code functionality for account recovery
- Azure SQL Database with managed identity integration for secure, token-based access.

Findings

CSP



Content Security Policy

[Sample unsafe policy](#) [Sample safe policy](#)

```
default-src 'self'; script-src 'self' 'unsafe-inline'; style-src 'self' 'unsafe-inline'; img-src 'self' data:
https://api.qrserver.com; connect-src 'self' https://*.blob.core.windows.net https://*.microsoftonline.com
https://login.microsoft.com https://maanit-func.azurewebsites.net]
```

Evaluated CSP as seen by a browser supporting CSP Version 3

[expand/collapse all](#)

✓ default-src		✓
❗ script-src		^
⚠ 'self'	'self' can be problematic if you host JSONP, AngularJS or user uploaded files.	
❗ 'unsafe-inline'	'unsafe-inline' allows the execution of unsafe in-page scripts and event handlers.	
✓ style-src		✓
✓ img-src		✓
✓ connect-src		✓
ⓘ require-trusted-types-for [missing]	Consider requiring Trusted Types for scripts to lock down DOM XSS injection sinks. You can do this by adding "require-trusted-types-for 'script'" to your policy.	✓

Key Vulnerabilities

- Improperly configured Content Security Policy with permissive directives
- CSP includes 'unsafe-inline' for script-src and style-src
- Missing protection mechanisms such as nonces or hashes for inline scripts

Remediation

- Remove 'unsafe-inline' directives from script-src and style-src
- Implement nonce-based or hash-based CSP for necessary inline scripts
- Add strict-dynamic to enhance protection for script loading

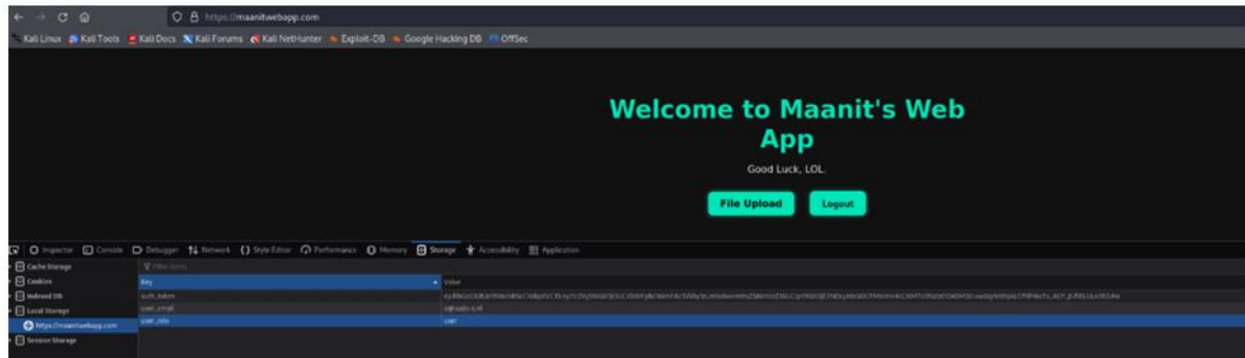
- Implement `require-trusted-types-for` to prevent DOM XSS
- Set `frame-ancestors` directive to prevent clickjacking
- Configure `report-uri/report-to` for CSP violation monitoring
- Consider using CSP Level 3 features for improved security
- Regularly test CSP configuration with tools like Google's CSP Evaluator

[illegible]

- 2FA validation endpoint lacks rate limiting protection
- Endpoint reveals which users have 2FA enabled (information disclosure)
- User IDs can be enumerated by changing parameters in API requests
- Direct access to 2FA verification without prior authentication steps
- 9-character verification codes vulnerable to brute force attacks

- Implement strict rate limiting on 2FA validation endpoints
- Add progressive delays or account lockouts after multiple failed attempts
- Require prior successful password authentication before allowing 2FA attempts
- Use secure, non-sequential user identifiers to prevent enumeration
- Increase 2FA code complexity or implement time-based limitations
- Add monitoring and alerting for unusual 2FA validation patterns
- Implement proper authentication flow that prevents skipping steps
- Consider implementing multi-channel 2FA (email + SMS or app) for critical accounts

Insecure Token Storage and Client-Side Role Validation



```
if (token) {
    debugLog('Found JWT in localStorage');

    // Attempt to show authenticated UI elements
    if(logoutButton) logoutButton.style.display = 'inline-block';
    if(loginLink) loginLink.style.display = 'none';
    if(registerLink) registerLink.style.display = 'none';

    try {
        const parts = token.split('.');
        if (parts.length === 3) {
            const payload = JSON.parse(atob(parts[1])); // Decode the payload
            debugLog(`Token payload: userId=${payload.userId}, email=${payload.email}, role=${payload.role || 'none'}`);

            // Store/update user info in localStorage from the token's payload
            // This ensures consistency if other pages (like login/2fa-verify) didn't set them fully.
            if (payload.email) {
                localStorage.setItem('user_email', payload.email);
            } else {
                localStorage.removeItem('user_email'); // Clear if not present
            }

            if (payload.role) {
                localStorage.setItem('user_role', payload.role); // Set role from JWT
            } else {
                localStorage.removeItem('user_role'); // Clear role if not in token
                debugLog('Role not found in token payload.');
            }

            // Check if the role from the token is admin
            if (payload.role && payload.role.toLowerCase() === 'admin') {
                debugLog('Admin role detected in token, showing admin links');
                if(adminPanellink) adminPanellink.style.display = 'inline-block';
                if(userslink) userslink.style.display = 'inline-block';
            } else {
                // This else is important to explicitly hide admin links if not admin
                debugLog('Not an admin role, ensuring admin links are hidden.');
```

```
// File selection handler
fileInput.addEventListener('change', function() {
  if (this.files.length === 0) {
    filenameDisplay.textContent = 'No file selected';
    fileDetails.style.display = 'none';
    uploadButton.disabled = true;
    return;
  }

  const file = this.files[0];
  filenameDisplay.textContent = file.name;
  fileSize.textContent = formatFileSize(file.size);
  fileType.textContent = file.type || 'Unknown';
  fileDetails.style.display = 'block';

  // Validate file type and size
  const validExtensions = ['.pdf', '.doc', '.docx', '.txt', '.jpg', '.jpeg', '.png', '.gif', '.csv', '.json', '.xml'];
  const maxSize = 10 * 1024 * 1024; // 10MB

  const fileExtension = '.' + file.name.split('.').pop().toLowerCase();
  const isValidType = validExtensions.includes(fileExtension);
  const isValidSize = file.size <= maxSize;

  // Also validate content type for security
  const validMimeTypes = [
    'application/pdf',
    'application/msword',
    'application/vnd.openxmlformats-officedocument.wordprocessingml.document',
    'text/plain',
    'image/jpeg',
    'image/png',
    'image/gif',
    'text/csv',
    'application/json',
    'application/xml',
    'text/xml'
  ];

  const isValidMimeType = !file.type || validMimeTypes.includes(file.type);

```

Key Vulnerabilities

- Authentication tokens stored in browser localStorage
- User role claims from JWT processed on client-side for UI rendering
- Client-side validation of authorization roles instead of server-controlled rendering
- JWT payload directly exposed to JavaScript environment

Remediation

- Store authentication tokens in HTTP-only cookies with Secure and SameSite flags
- Implement server-side rendering based on authenticated user permissions
- Remove role information from client-visible JWT payload or use separate tokens
- Implement proper server-side validation of all JWT signatures and claims
- Add automatic token rotation and shorter expiration timeframes
- Consider implementing token binding to prevent token theft and reuse
- Strictly separate UI rendering logic from authorization decisions

- Use server-driven UI state to prevent client-side tampering with interface elements

File Upload Vulnerability

[illegible]

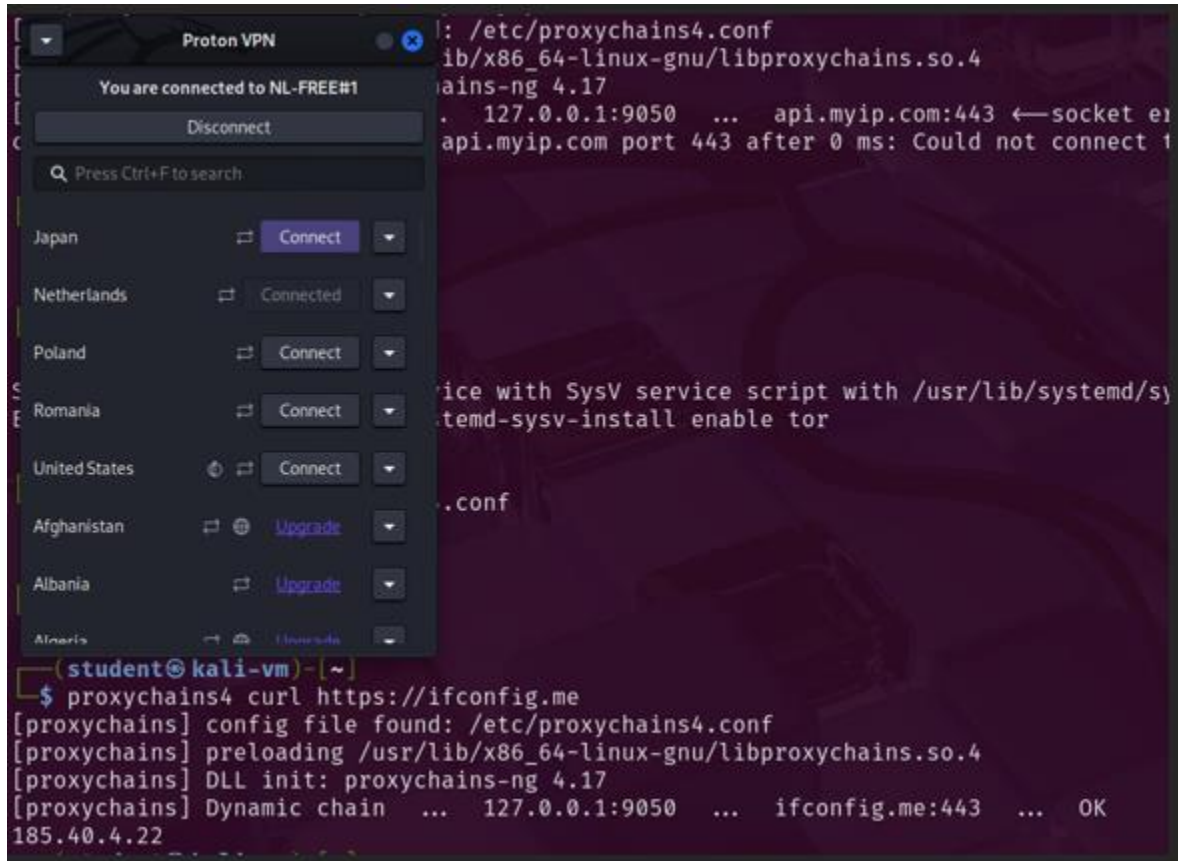
Key Vulnerabilities

- File upload security relies solely on client-side validation
- Content-Type header can be modified to bypass MIME type restrictions
- blobName parameter can be altered to change file extensions
- SAS token mechanism lacks validation of the requested file properties

Remediation

- Implement comprehensive server-side validation of all uploaded files
- Verify file content using magic bytes inspection rather than trusting headers
- Generate random filenames and store original names separately in a database
- Configure Azure Blob Storage permissions to prevent execution of code
- Scan file content for malicious patterns before accepting uploads
- Implement proper Content-Type enforcement on the server side
- Store uploads in containers with appropriate access control and no execute permissions
- Validate both file extension and content before generating SAS tokens

Rate Limiting Bypass



Key Vulnerabilities

- No detection or filtering of traffic from VPN or TOR exit nodes
- Rate limiting controls can be bypassed by changing IP addresses
- Security monitoring fails to identify anonymized access patterns

Remediation

- Implement IP intelligence services to identify and classify traffic from anonymizing networks
- Configure Azure WAF to detect and apply stricter controls to VPN/TOR traffic
- Deploy CAPTCHA or other human verification for sensitive operations
- Implement client fingerprinting beyond IP address for rate limiting
- Create risk-based authentication that considers connection source as a factor

- Set up alerts for access patterns associated with anonymizing networks
- Consider requiring verified email or phone authentication for registration
- Implement progressive security measures based on threat intelligence

Poorly Configured WAF

```
(student@kali-vm)-[~]
$ proxychains4 wafw00f https://maanitwebapp.com
[proxychains] config file found: /etc/proxychains4.conf
[proxychains] preloading /usr/lib/x86_64-linux-gnu/libproxychains.so.4
[proxychains] DLL init: proxychains-ng 4.17
This request is not authorized to perform this operation using this permission. RequestId:82331a4d-e01e-0038-b38c-c5d809000000
~ WAFW00F ~
~ WAFW00F : v2.3.1 ~
The Web Application Firewall Fingerprinting Toolkit
[*] Checking https://maanitwebapp.com
[proxychains] Dynamic chain ... 127.0.0.1:9050 ... maanitwebapp.com:443 ... OK
[proxychains] Dynamic chain ... 127.0.0.1:9050 ... maanitwebapp.com:443 ... OK
[+] The site https://maanitwebapp.com is behind Azure Front Door (Microsoft) WAF.
[~] Number of requests: 2
```

Key Vulnerabilities

- Azure Front Door WAF fails to block wildcard characters in input fields
- Special characters pass through without validation or sanitization
- WAF operates in detection mode rather than prevention mode
- Pattern matching for common attack vectors is insufficient

Remediation

- Configure Azure Front Door WAF in prevention mode rather than detection mode
- Enable OWASP ModSecurity Core Rule Set (CRS) 3.3+ with appropriate sensitivity
- Implement custom WAF rules to block malformed inputs and wildcard characters
- Create specific rules for known attack patterns based on web application functionality
- Regularly test WAF rules with known payload samples to verify effectiveness
- Enable comprehensive WAF logging for security analysis
- Consider implementing additional layers of input validation at the application level

- Add rate limiting for requests containing suspicious character sequences

Brute Force Bypass

```
(student@kali-vm)-[~]
$ proxychains4 gobuster dir -u https://maanitwebapp.com/api -w /usr/share/wordlists/dirbuster/directory-list-lowercase-2.3-small.txt -r -f -k -a "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/122.0.0.0 Safari/537.36" --status-codes-blacklist 403 -o gobuster_results.txt
[proxychains] config file found: /etc/proxychains4.conf
[proxychains] preloading /usr/lib/x86_64-linux-gnu/libproxychains.so.4
[proxychains] DLL init: proxychains-ng 4.17

Gobuster v3.6
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)

[+] Url:             https://maanitwebapp.com/api
[+] Method:          GET
[+] Threads:         5
[+] Wordlist:         /usr/share/wordlists/dirbuster/directory-list-lowercase-2.3-small.txt
[+] Negative Status codes: 403
[+] Exclude Length:  6345
[+] User Agent:      Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/122.0.0.0 Safari/537.36
[+] Add Slash:       true
[+] Follow Redirect: true
[+] Timeout:         10s

Starting gobuster in directory enumeration mode

/users/ (Status: 500) [Size: 94]
Progress: 1705 / 81644 (2.09%)^C
[!] Keyboard interrupt detected, terminating.
Progress: 1711 / 81644 (2.10%)

Finished
```

Key Vulnerabilities

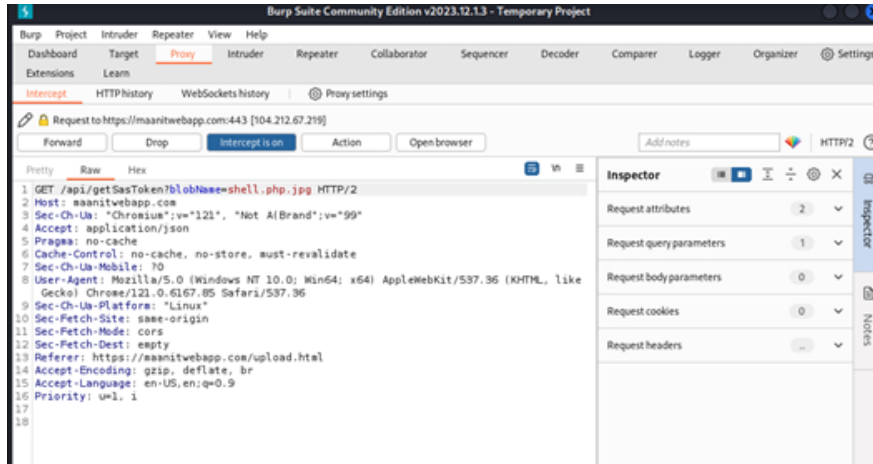
- Detection systems fail to identify directory enumeration attempts performed at slow rates
- WAF and IDS mechanisms only trigger on high-frequency request patterns
- Status code and response length filtering allows attackers to identify valid endpoints
- No cumulative tracking of low-rate scanning activity over extended periods

Remediation

- Implement behavioral analysis that considers request patterns over extended timeframes
- Configure detection systems to track cumulative scanning attempts, not just request frequency
- Deploy machine learning-based anomaly detection for identifying unusual access patterns
- Implement consistent response times and sizes for all endpoints, whether valid or invalid
- Add honeypot endpoints to detect scanning activities and trigger alerts
- Monitor and analyze specific user agent patterns typical of scanning tools
- Implement progressive throttling based on cumulative access patterns

- Consider using a web application firewall with more sophisticated pattern recognition capabilities

Bypassing File Upload Rules



Key Vulnerabilities

- PHP shell with disguised file extension (.php.jpg) successfully uploaded
- Client-side file validation bypassed using curl and direct API requests
- SAS token generation endpoint allows arbitrary file uploads
- No server-side content validation or file type verification

Remediation

- Implement server-side file content validation using magic bytes inspection
- Analyze actual file content rather than relying on extension or Content-Type
- Store uploaded files in non-executable directories with appropriate permissions
- Configure web server to not process PHP or other executable file types in upload directories
- Rename files on upload with random names and maintain original name in database
- Apply principle of least privilege to upload storage locations
- Scan uploaded files with antivirus/anti-malware before storing
- Implement file size restrictions and content validation based on expected file types.